CSC1029 OO Programming

Lecture 21 – Searching Data

QUEEN'S UNIVERSITY BELFAST

What's to come in todays lecture?

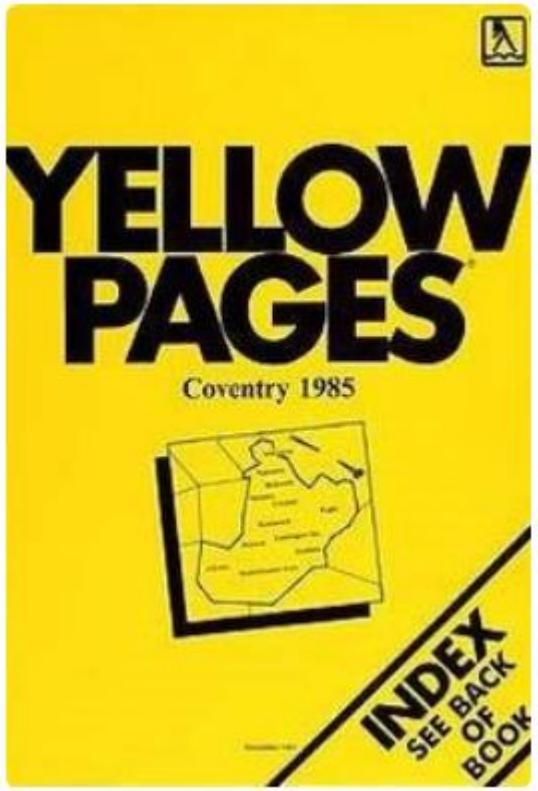
- An Overview of Searching Algorithms
- Searching Unordered Data
- Searching Ordered Data

Searching for Information (1)

How do you look for something in a telephone directory?

What ***input*** is required for the search process?

What ***output*** is produced from the search process?



Search Criteria

Data to Search

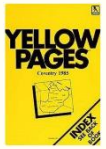
Search Algorithm

Search Result

Searching for Information (2)

What would your ***Search Algorithm*** look like?

#1. Search Criteria: Queen's University

#2. Data to Search: 

A Simple Algorithm:

```
Open the book
Skip to entries starting with 'Q'
Further skip to entries starting with
'Queen'
Isolate entry for 'Queen's University'
```

Search Result:

Either you will find what you are looking for: 02890245133

… or you won't, if it does not exist.

Is this an efficient way to search?

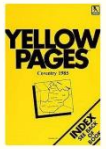
What does this efficiency depend on?

How would this efficiency be measured?

Searching for Information (3)

What would your ***Search Algorithm*** look like if the search data is not sorted?

#1. Search Criteria: Queen's University

#2. Data to Search: 

A Simple Algorithm:

```
Open the book
Start with the first entry
Repeat
   if this entry is a match
      stop as it's been found
   get the next entry
Until no entries left
```

Search Result:

Either you will find what you are looking for: 02890245133

... or you won't, if it does not exist.

Is this an efficient way to search?

How would this efficiency be measured?

Searching for Information (4)

What would your ***Search Algorithm*** look like if the search data is not sorted?

Searching through data that is ***not organised or sorted*** will typically require a ***linear search***.

You may need to search through ***all*** entries to find what you are looking for.

You may need to search through all entries and come up with nothing!

Linear (or Sequential) Search:

A method for finding an element within a list, ***sequentially*** checking each element of the list until a match is found or the whole list has been searched.

Best Case: element at start of list – only 1 comparison is required

Worst Case: element at end of list of size n (n comparisons are required)

Average Case: for a list of size n (n/2 comparisons are required)

Linear or Sequential Searching (1)

```
public static void main(String[] args) {
   int values[] = {3, 1, 6, 5, 11, 7, 2};
   int num = 6;

   boolean found = search(values, num);

   System.out.print("Number: "+num);
   if ( found ) {
      System.out.println(" was found.");
   }
   else {
      System.out.println(" was not found.");
   }

}
```

```
public static boolean search(int data[],
                             int target) {

   for(int i=0; i<data.length; i++) {
      if ( data[i] == target ) {
         return true;
      }
   }
   return false;
}
```

```
Number: 6 was found.
```

Linear or Sequential Searching (2)

```
public static void main(String[] args) {
   int values[] = {3, 1, 6, 5, 11, 7, 2};
   int num = 6;

   int pos = seekPosition(values, 6);
   System.out.print("Number: "+num);
   if ( pos >= 0) {
      System.out.println(" was found at
                              position: "+pos);
   }
   else {
      System.out.println(" was not found.");
   }

}
```

```
public static int seekPosition(int data[],
                               int target) {
   for(int i=0; i<data.length; i++) {
      if ( data[i] == target ) {
         return i;
      }
   }
   return -1;
}
```

```
Number: 6 was found at position: 2
```

QUEEN'S UNIVERSITY BELFAST

Linear or Sequential Searching (3)

```
public static int findMax(int values[]) {
   int max = values[0];
   for(int i=1; i<values.length; i++) {
      if (values[i] > max) {
         max = values[i];
      }
   }
   return max;
}
```

```
public static int findMin(int values[]) {
   int min = values[0];
   for(int i=1; i<values.length; i++) {
      if (values[i] < min) {
         min = values[i];
      }
   }
   return min;
```

```
public static void main(String[] args) {
   int values[] = {3, 1, 6, 5, 11, 7, 2};
   int num = 6;


   int max = findMax(values);
   int min = findMin(values);
   System.out.println("Max value is: " +
                              max);
   System.out.println("Min value is: " +
                              min);

}
```

```
Max value is: 11
Min value is: 1
```

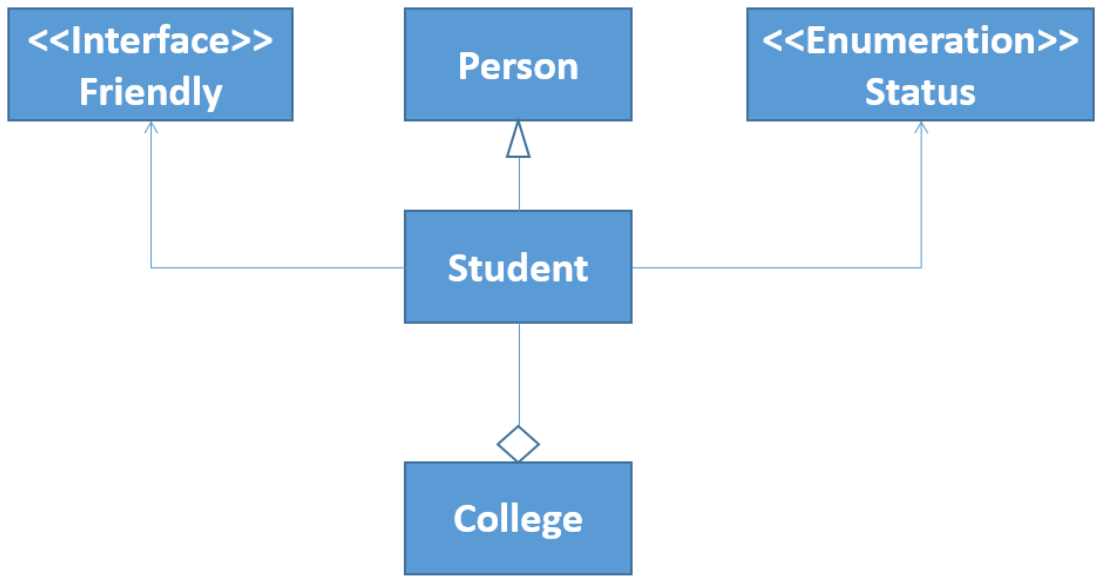
Linear or Sequential Searching (4)

We can extend the linear search algorithm to use objects …

In the previous lectures we defined a class ***Student***

We could consider a larger system where student objects are managed through a ***College*** …

… which contains an ***ArrayList*** (of Student) called ***register***.



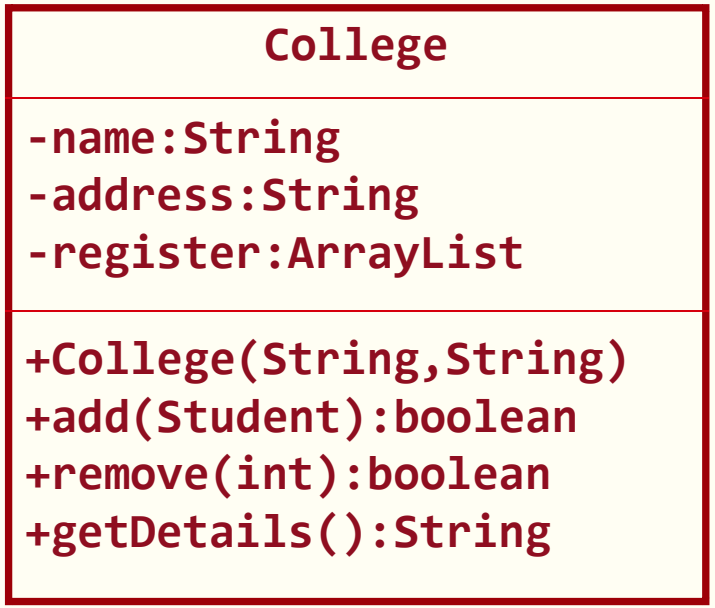
Linear or Sequential Searching (5)

We can extend the linear search algorithm to use objects …

In the previous lectures we defined a class ***Student***

We could consider a larger system where student objects are managed through a ***College*** …

… which contains an ***ArrayList*** (of Student) called ***register***.

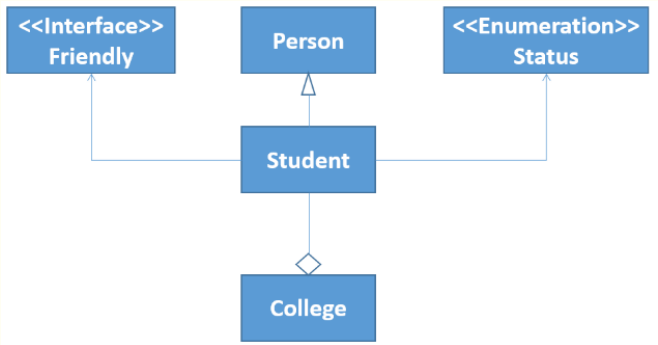


How would the ***remove*** method be implemented?

This would require a ***search*** method …

Linear or Sequential Searching (6)

College
-name:String -address:String -register:ArrayList
+College(String,String) +add(Student):boolean +remove(int):boolean +getDetails():String




```
// removing a student from the register
// based on student id
public boolean remove(int id) {
    Student stu = search(id);
    if ( stu != null ) {
        return register.remove(stu);
    }
    return false;
}
```

```
public Student search(int id) {
    ...
}
```

QUEEN'S UNIVERSITY BELFAST

Linear or Sequential Searching (7)

```
/**
 * Search for a student with specific id in register
 * @param id - a unique student id
 * @return - a reference to the Student (if found)
 * and null otherwise
 */
public Student search(int id) {
   Student target = null;
   for(int index=0;index<register.size();index++) {
      Student stu = register.get(index);
      if ( stu.getSNumber() == id ) {
         target = stu;
         break;
      }
   }
   return target;
}
```

Linear or Sequential Searching (8)

```
public static void main(String[] args) {
   College myCollege = initialise();

   // output college details
   System.out.println(myCollege.getDetails());

   // search for and display details of a student
   int id = 2;
   System.out.println("Search for student with id:
                                    "+id);
   Student stu = myCollege.search(id);
   if ( stu != null ) {
      System.out.println(stu.getDetails());
   }
   else {
      System.out.println("Student with id: "+id+"
                     does not exist.");
   }
}
```

```
private static College initialise() {
   College aCollege = new College("QUB",
                                  "18 Malone Road");
   Student stu1 = new Student("Homer", "Belfast",
                                  Status.GRADUATE);
   Student stu2 = new Student("Marge", "Belfast",
                                  Status.GRADUATE);
   aCollege.add(stu1);
   aCollege.add(stu2);
   return aCollege;
}
```

```
Search for student with id: 2
Name: Marge, Address: Belfast
Number: 2
Status: Status: Graduate
```

QUEEN'S UNIVERSITY BELFAST

Linear or Sequential Searching (9)

We can extend the linear search algorithm to use objects …

In the previous lectures we defined a class *Student*

We could consider a larger system where student objects are managed through a *College* …

… which contains an *ArrayList* (of Student) called *register*.

```
public ArrayList<Student> search(String name) {
    ArrayList<Student> result = new
                                    ArrayList<Student>();
    for(Student stu : register) {
        if ( stu.getName().contains(name)) {
            result.add(stu);
        }
    }
    return result;
}
```

String method – returns *true* if a string contains a specified character sequence and *false* otherwise.

Linear or Sequential Searching (10)

We can extend the linear search algorithm to use objects …

In the previous lectures we defined a class ***Student***

We could consider a larger system where student objects are managed through a ***College*** …

… which contains an ***ArrayList*** (of Student) called ***register***.

```
// search for students by name
String aName = "Homer";
ArrayList<Student> data = myCollege.search(aName);
System.out.println("\nStudents with name: "+
                                        aName+" are:");
if ( data.size() == 0) {
   System.out.println("None.");
}
else {
   for(Student s : data) {
      System.out.println(s.getDetails());
   }
}
```

```
Students with name: Homer are:
Name: Homer Simpson, Address: Belfast
Number: 1
Status: Status: Graduate
```

Linear or Sequential Searching (11)

This technique is ***not efficient***.

It's only practical for very ***small amounts*** of data.

Alternative approaches will require that the ***data is sorted*** in some way before searching.

This will result in a more rapid location (or not) of search data

This alternative approach will be considered in the next lecture.

What's Next?

Searching Ordered Data